

Lab Manual

Session-2022

Semester Fall 2023

Course: CS-271L Computer Organization and Assembly Language

Instructor: Mr. Nadeem Iqbal



Department of Computer Science
University of Engineering & Technology Lahore. (KSK Campus)

Table of Contents

Lab Number. 1	3
AIM: Introduction to Assembly Programming	3
Lab Number. 2	16
AIM: Write an assembly language program to perform 8-bit and 16-bit addition.....	16
Lab Number. 3	22
AIM: Arrays implementation	22
Lab Number. 4	27
AIM: Conditional execution using Jmp and Loop instructions	27
Lab Number. 5	32
AIM: To display a Character, Read character input, Display String, Data movement and arithmetic instruction	32
Lab Number. 6	34
AIM: PUSH and POP instructions, Procedure implementation, Register parameters	34
Lab Number. 7	38
AIM: Understand Procedures and Boolean Instructions.....	38
Lab Number. 8	45
AIM: Understand Shift and Rotate Instructions, MUL, IMUL, DIV, IDIV Instructions.	45
Lab Number. 9	52
AIM: To perform Extended Addition and Subtraction, ASCII and Unpacked Decimal Arithmetic and Packed Decimal Arithmetic	52
Lab Number. 10	55
AIM: Advanced Procedures.....	55
Lab Number. 11	60
AIM: Creating Multimodule Programs.....	60
Lab Number. 12	70
AIM: Structures and Macros.....	70
Lab Number. 13	73
AIM: MS- DOS/ IO service Routine, INT Instruction	73

Lab Number. 1

AIM: Introduction to Assembly Programming

CLO: 1	
--------	--

No evaluation done in First Lab

Objectives:

- To understand the working of EMU 8086 as emulator for 8086 Programming and get acquainted with Assembly Language Programming.

Theory:

Assembly Language Programming is less powerful than the high-level languages like C, C++, JAVA, etc. it is necessary to understand basics of assembly language as it helps in understanding the working and the instruction set of microprocessors and microcontrollers. Microprocessors like 8085, 8086 and many other microcontrollers could be easily operated via simple instructions of assembly languages. MASM (Microsoft Macro Assembler) is a very efficient assembly language programming tool for windows and MS-DOS. It is not an emulator, but an actual programming tool helps in programming with processor. Emulator EMU8086 is an emulator for 8086 providing an easy and user-friendly environment for assembly language programming for 8086.

INTRODUCTION TO ASSEMBLY LANGUAGE PROGRAMMING: LEVELS OF PROGRAMMING:

There are three levels of programming

1. **Machine language** programs are programs that the computer can understand and execute directly.
2. **Assembly language** instructions match machine language instructions but are written using character strings so that they are more easily understood.
3. **High-level language** instructions are much closer to the English language and are structured.

Ultimately, an assembly language or high-level language program must be converted into machine language by programs called translators. If the program being translated is in assembly language, the translator is referred to as an assembler, and if it is in a high-level language the translator is referred to as a compiler or interpreter.

ASSEMBLY LANGUAGE PROGRAM DEVELOPMENT TOOLS:

Assembler

It is a system program which converts the assembly language program instructions into machine executable instructions. For example: Microsoft Macro Assembler (MASM), Borland Turbo Assembler (TASM), Open Source Netwide Assembler (NASM) etc.

- **MASM**

The Microsoft Macro Assembler (MASM) is an x86 assembler for MS-DOS and Microsoft Windows. It supports a wide variety of macro facilities and structured programming idioms, including high-level functions for looping and procedures. Later versions added the capability of producing programs for Windows.

- **TASM**

Turbo Assembler (TASM) is an x86 assembler package developed by Borland. It is used with Borland's high-level language compilers, such as Turbo Pascal, Turbo Basic and Turbo C. The Turbo Assembler package is bundled with the linker, Turbo Linker, and is interoperable with the Turbo Debugger.

- **NASM**

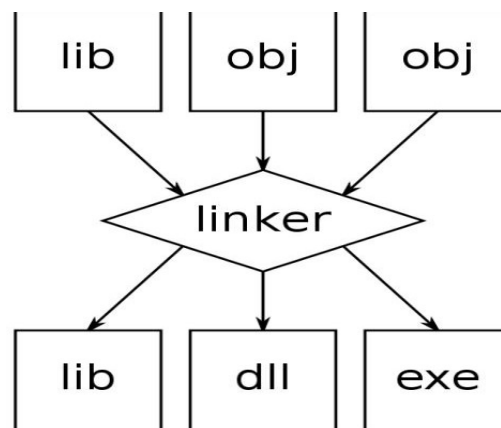
The Net wide Assembler (NASM) is an assembler and disassembler for the Intel x86 architecture. It can be used to write 16-bit, 32-bit (IA-32) and 64-bit (x86-64) programs. NASM is considered to be one of the most popular assemblers for Linux and is the second most popular assembler overall, behind MASM. NASM was originally written by Simon Tatham with assistance from Julian Hall and is currently maintained by a small team led by H. Peter Anvin.

Emulators

Emulator allows computer program to run on a platform (computer architecture and/or operating system) other than for which they were originally developed unlike the simulation which only attempts to reproduce a program's behavior, emulation attempts to model to various degrees the state of device being emulated. EMU8086 is emulator which emulates the behavior of 8086 machine. EMU8086 IDE contains built in assembler, which is compatible with syntax of MASAM, TASAM and few other assemblers. It also contains debugger which can be used to debug executable program. In EMU8086 one can emulate each instruction of program one by one and can views the value of registers, flags and variable changed by instruction.

Linker

Linker or link editor is a program that takes one or more objects generated by a compiler and combines them into a single executable program. When a program comprises multiple object files, the linker combines these files into a unified executable program. Linkers can take objects from a collection called a library. Some linkers do not include the whole library in the output; they only include its symbols that are referenced from other object files or libraries.



Debugger

A debugger is a program that allows you to trace the execution of a program and examine the content of registers and memory.

- For 16-bit programs, MASM supplies a 16-bit debugger named CodeView. CodeView can be used to debug only 16-bit programs and is already provided with the MASM 6.15 distribution.

Editor

You need a text editor to create assembly language source files. MASM6.15 has its own editor or you can use for example Notepad++. To make programs in assembly language, you must know some information about the 8086 microprocessor. The 8086 contains 14 registers. Each register is 16 bits long.

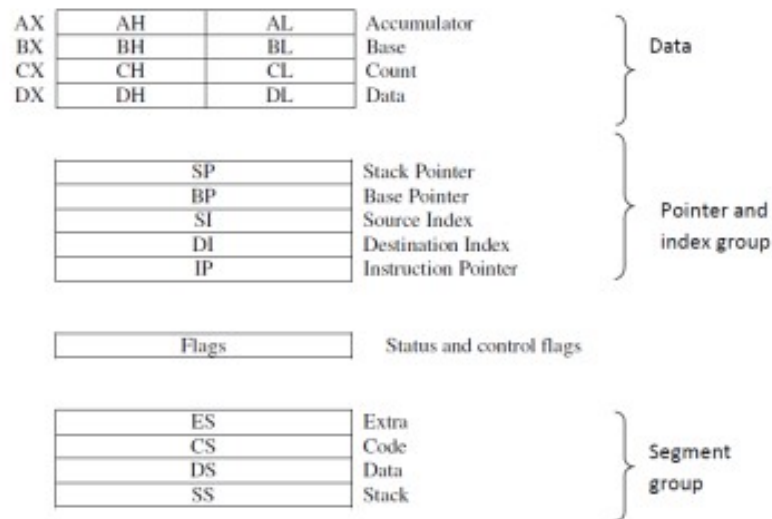
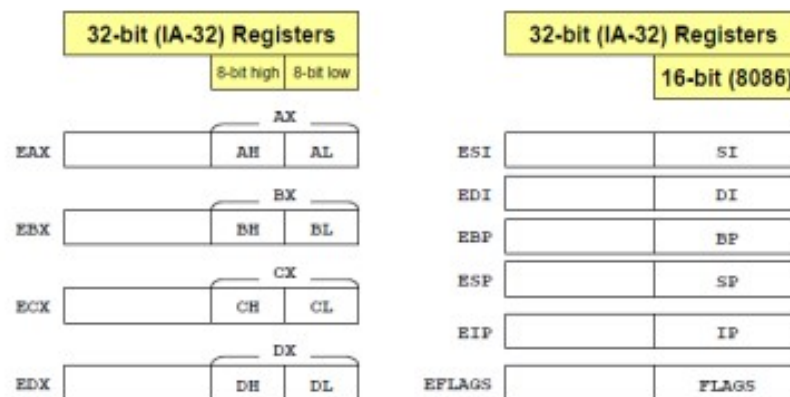
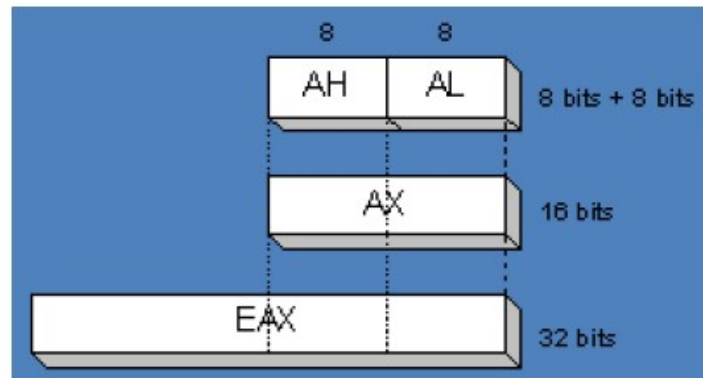


Figure (1): Registers of 8086 microprocessor



General-Purpose Registers:

Named storage locations inside the CPU, optimized for speed. To Access Parts of Registers we can Use 8-bit name, 16-bit name, or 32-bit name; this is applied to EAX, EBX, ECX, and EDX.



Each register has different usage as shown in Table (1) below. The general-purpose registers can be "split". AH contains the high byte of AX and AL contains the low byte.

You also have: BH, BL, CH, CL, DL, DH.

So if for example, DX contains the value 1234h DH would be 12h and DL would be 34h.

Segment Registers		
CS	Code Segment	16-bit number that points to the active code-segment
DS	Data Segment	16-bit number that points to the active data-segment
SS	Stack Segment	16-bit number that points to the active stack-segment
ES	Extra Segment	16-bit number that points to the active extra-segment
Pointer Registers		
IP	Instruction Pointer	16-bit number that points to the offset of the next instruction
SP	Stack Pointer	16-bit number that points to the offset that the stack is using
BP	Base Pointer	used to pass data to and from the stack
General-Purpose Registers		
AX	Accumulator Register	mostly used for calculations and for input/output
BX	Base Register	Only register that can be used as an index
CX	Count Register	register used for the loop instruction
DX	Data Register	input/output and used by multiply and divide
Index Registers		
SI	Source Index	used by string operations as source
DI	Destination Index	used by string operations as destination

Table (1): Registers of 8086 microprocessor and their purposes

And a 16-bit FLAG Register. The FLAGS Register consists of 9 status bits. These bits are also called flags, because they can either be SET (1) or NOT SET (0). All these flags have a name and purpose.

Control Flags: Control flags control the CPU's operation. For example, they can cause the CPU to break after every instruction executes, interrupt when arithmetic overflow is detected. Programs can set individual bits in the EFLAGS register to control the CPU's operation.

Examples are the Direction, Trap and Interrupt flags.

Status Flags: The Status flags reflect the outcomes of arithmetic and logical operations performed by the CPU. They are the **Overflow, Sign, Zero, Auxiliary Carry, Parity, and Carry flags**

Abr.	Name	Description
OF	Overflow Flag	if <i>set</i> , an instruction generates an invalid signed result
DF	Direction Flag	used for string operations to check direction
IF	Interrupt Flag	if <i>set</i> , interrupt are enabled, else disabled
TF	Trap Flag	if <i>set</i> , CPU can work in single step mode
SF	Sign Flag	if <i>set</i> , resulting number of calculation is negative
ZF	Zero Flag	if <i>set</i> , resulting number of calculation is zero
AF	Auxiliary Carry	is set when an operation produces a carryout from bit 3 to bit 4
PF	Parity Flag	is set when an instruction generates an even number of 1 bits in the low byte of the destination operand.
CF	Carry Flag	is set when the result of an unsigned arithmetic operation is too large to fit into the destination.

MEMORY MODELS FOR THE ASSEMBLER

Model Type	Description
TINY	All data and code must fit into one segment. Tiny programs are written in .COM format, which means that the program must be originated at location 100H
SMALL	This model contains two segments: one data segment of 64K bytes and one code segment of 64K bytes.
MEDIUM	This model contains one data segment of 64K bytes and any number of code segments for large programs.
COMPACT	One code segment contains the program, and any number of data segments contains the data.
LARGE	The large model allows any number of code and data segments.
HUGE	This model is the same as large, but the data segments may contain more than 64K bytes each.
FLAT	Only available to MASM 6.X. The flat model uses one segment of 512K bytes to tear all data and code. Note that this model is mainly used with Windows NT

ASSEMBLER DIRECTIVES:

An assembler is a program used to convert an assembly language program into the equivalent machine code modules. The assembler decides the address of each label and substitutes the values

for each of the constants and variables. It then forms the machine code for mnemonics and data in assembly language program. Assembler directives help the assembler to correctly understand assembly language programs to prepare the codes. Commonly used assembler directives are DB, DD, DW, DUP, ASSUME, BYTE, SEGMENT, MACRO, PROC, OFFSET, NEAR, FAR, EQU, STRUC, PTR, END, ENDM, ENDP etc. Some directives generate and store information in the memory, while others do not.

DB: Define byte directive stores bytes of data in memory.

BYTE PTR: This directive indicates the size of data referenced by pointer.

SEGMENT: This directive is to indicate the start of the segment.

DUP (Duplicate): The DUP directive reserves memory locations given by the number preceding it but stores no specific values in any of these locations.

ASSUME: - The ASSUME statement is only used with full segment definitions. This statement tells the assembler what names have been chosen for the code, data, extra and stack segments.

EQU: - The equate directive equates a numeric ASCII or label to another label.

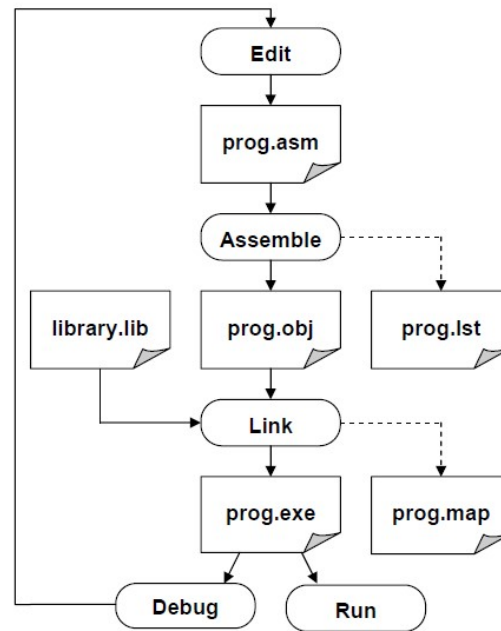
ORG: - The ORG (origin) statement changes the starting offset address in a segment.

PROC and ENDP: - The PROC and ENDP directives indicate start and end of a procedure (Sub routine). Both the PROC and ENDP directives require a label to indicate the name of the procedure. The PROC directive, must also be followed with the NEAR or FAR. A NEAR procedure is one that resides in the same code segment as the program. A FAR procedure may reside at any location in the memory system.

OFFSET: - Offset of a label. When the assembler comes across the OFFSET operator along with a label, it first computes the 16 – bit displacement of the particular label and replaces the string ‘OFFSET LABEL’ by the computed displacement.

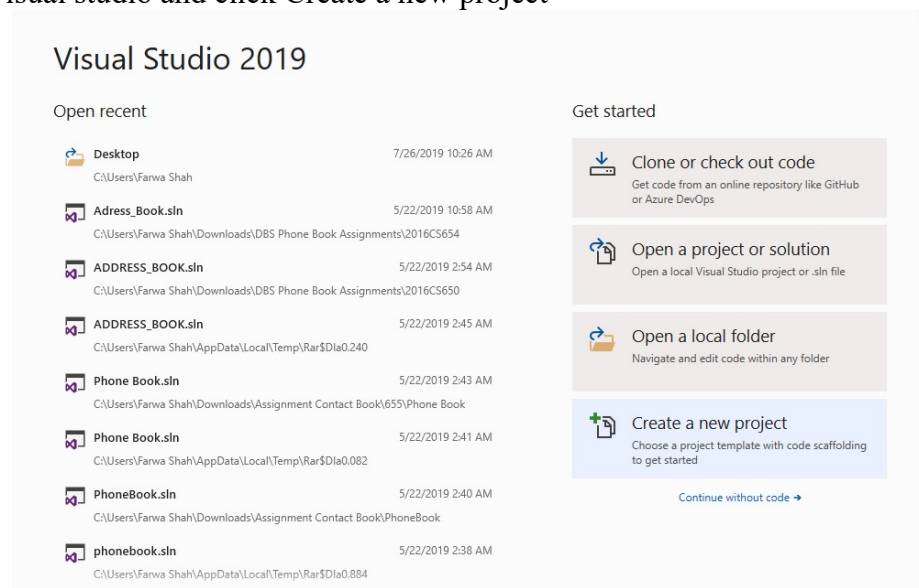
LENGTH: - Byte length of the label. This directive is used to refer to the length of data array or a string.

DOS Function Calls: In order to use DOS function calls, always place function number into register AH, and load other information into registers. Following is INT 21H, which is software interrupt to execute a DOS function. All function calls use INT 21H, and AH contains function call number. User can access the hardware of PC using DOS subroutine. DOS subroutines are invoked or called via software interrupt INT 21H.

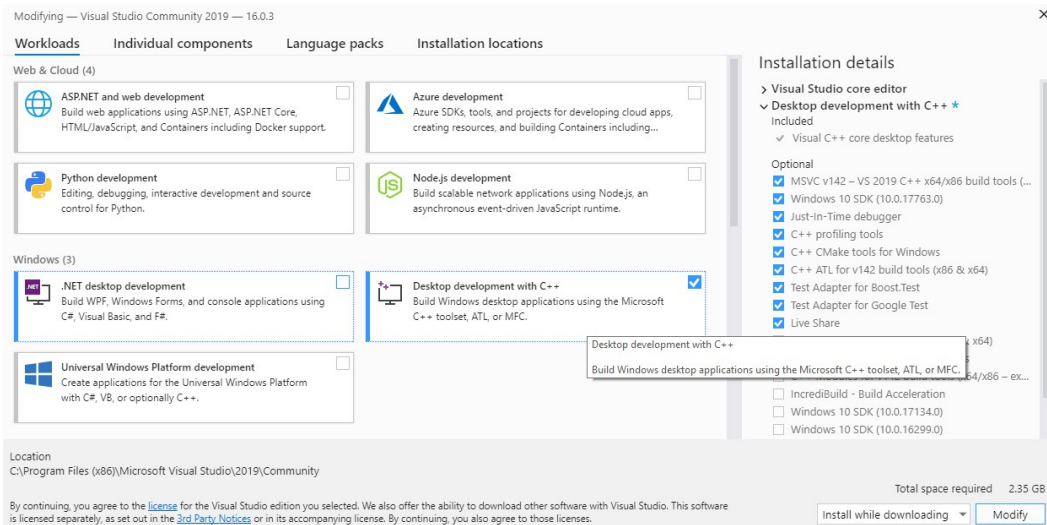


Configuration of visual studio 19 with assembly language

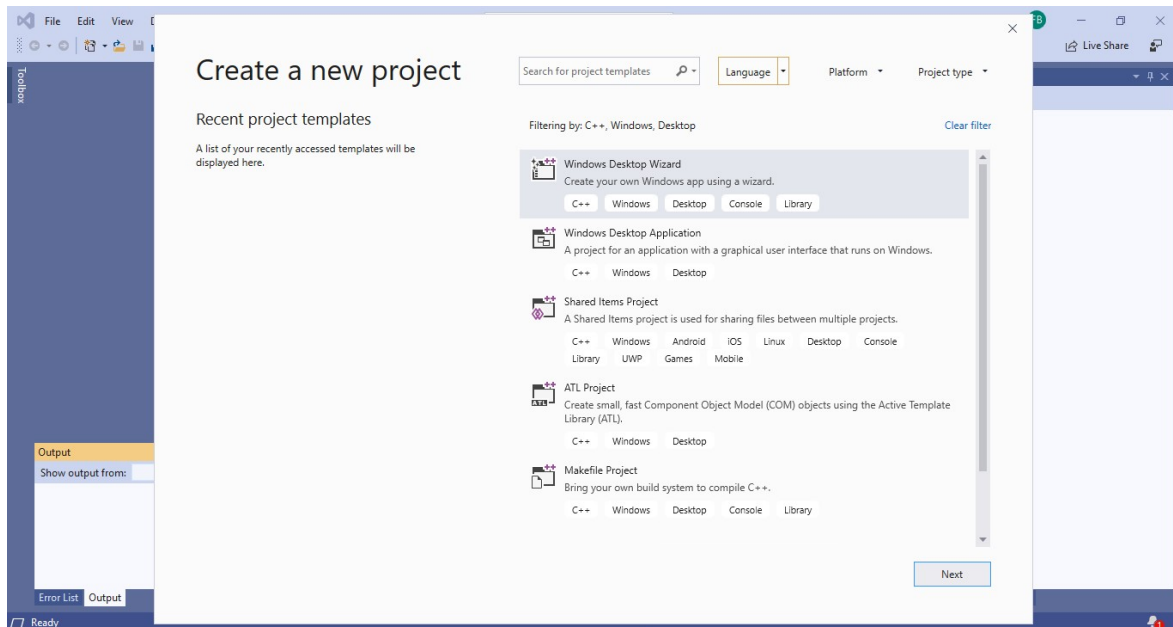
Step1: Open visual studio and click Create a new project



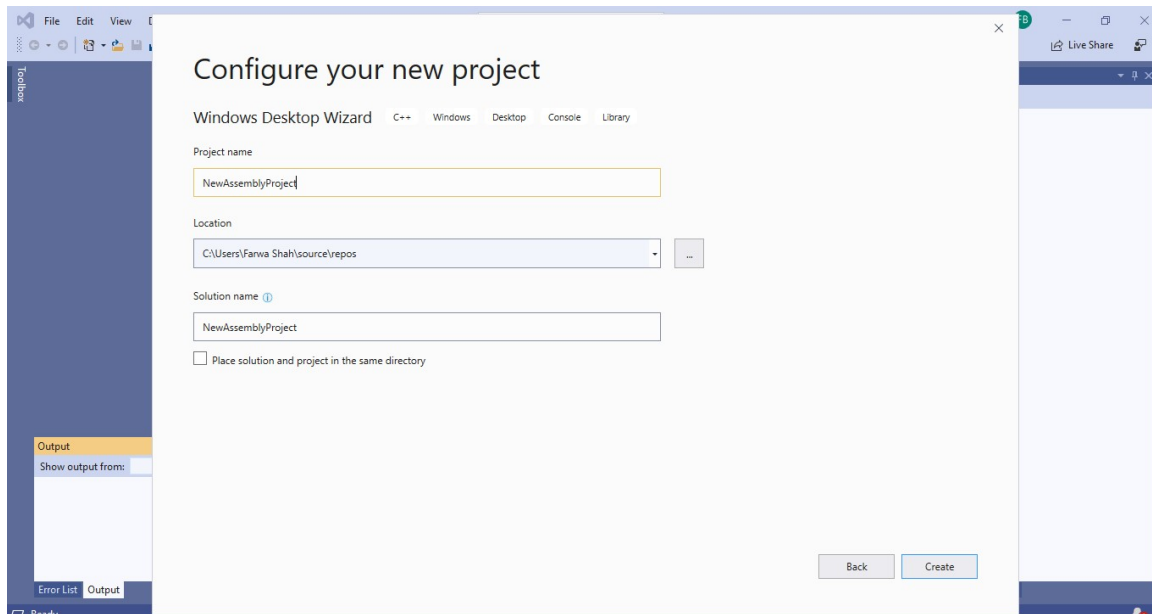
Step 2: From the top right select “C++” from Language, “Windows” from Platform and “Desktop” from Project Type. (Download the package if not present. Click on Desktop development with C++ and then Modify)



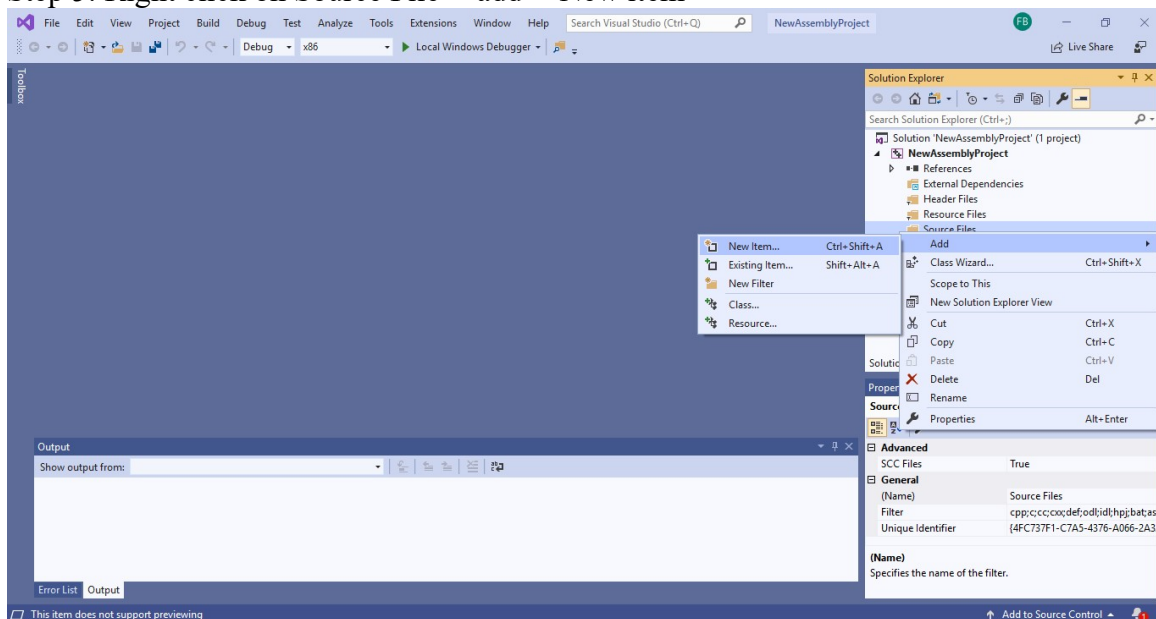
Step 3: After installing the package repeat step 2 and select Windows Desktop Wizard and then Next.



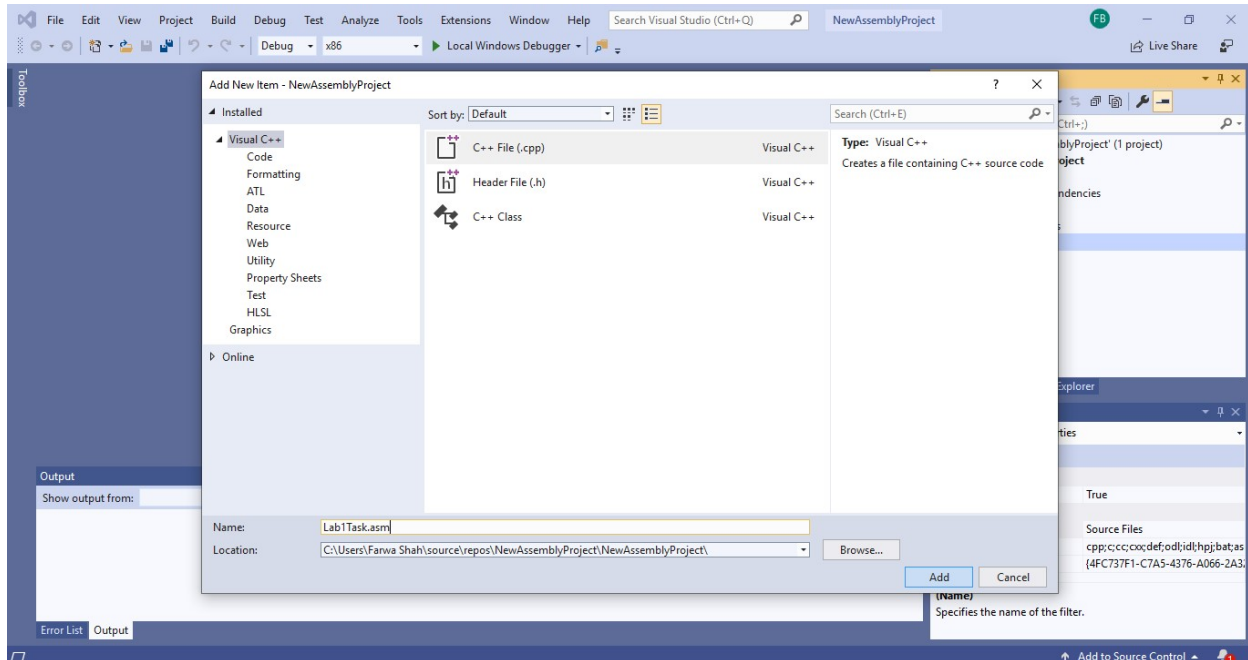
Step 4: Name your project name and create.



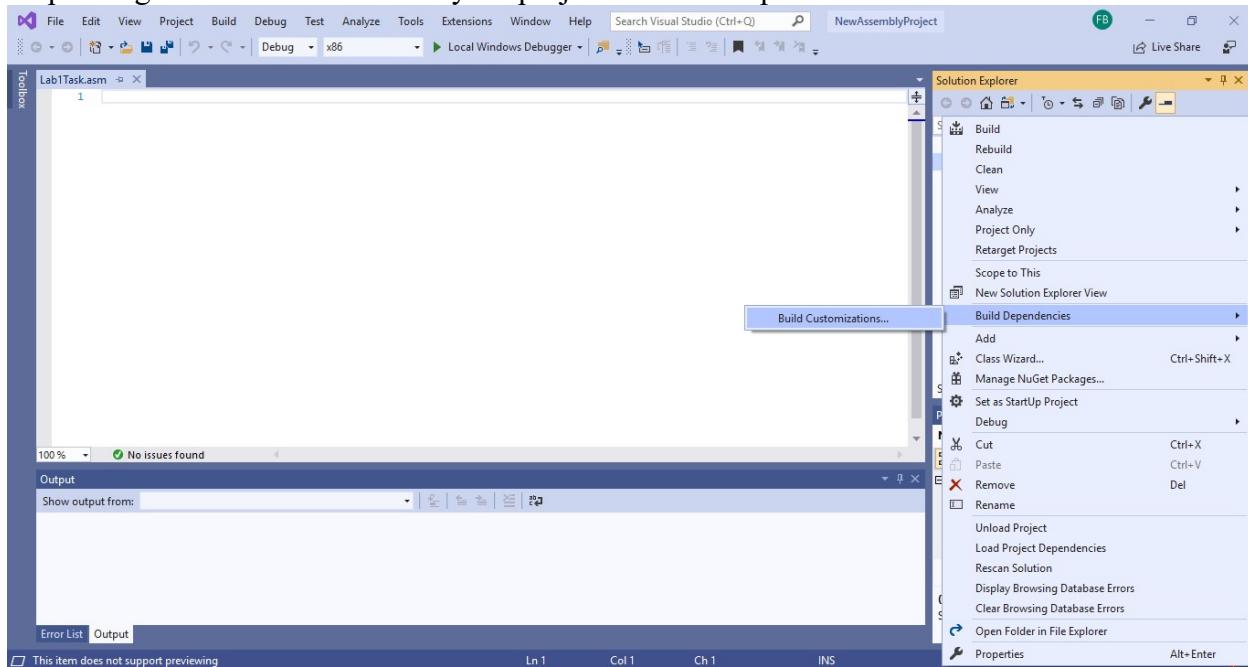
Step 5: Right click on Source File-> add-> New item



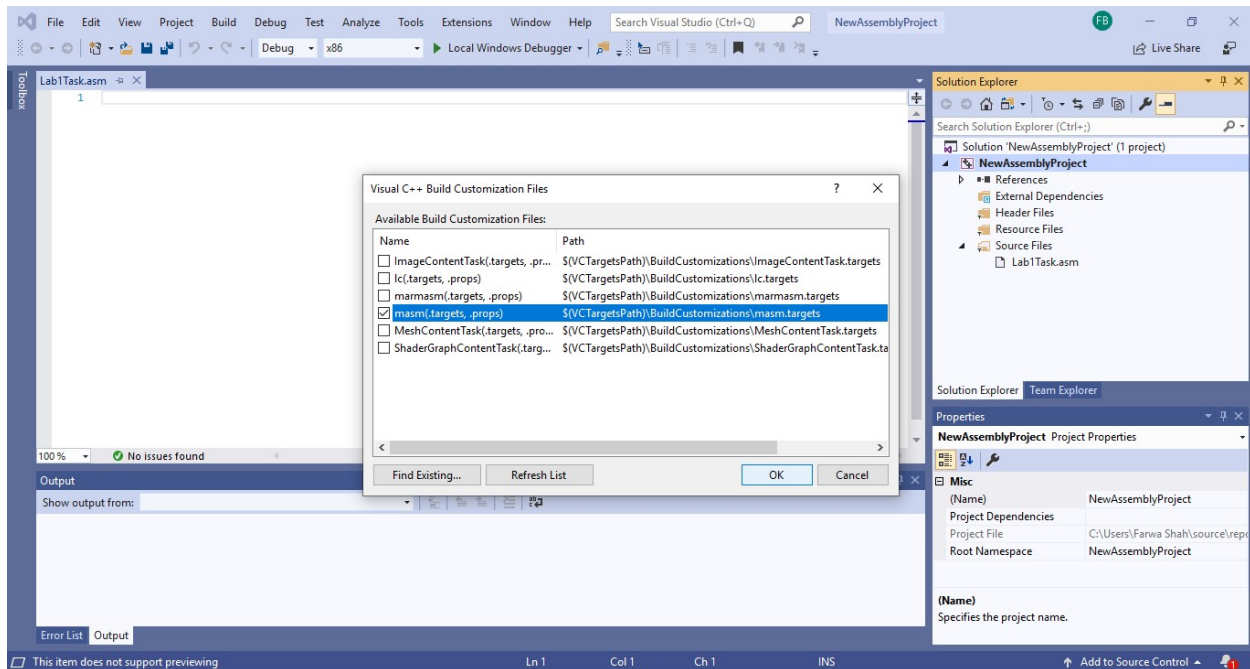
Step 6: Name your project with extension .asm, then click Add.



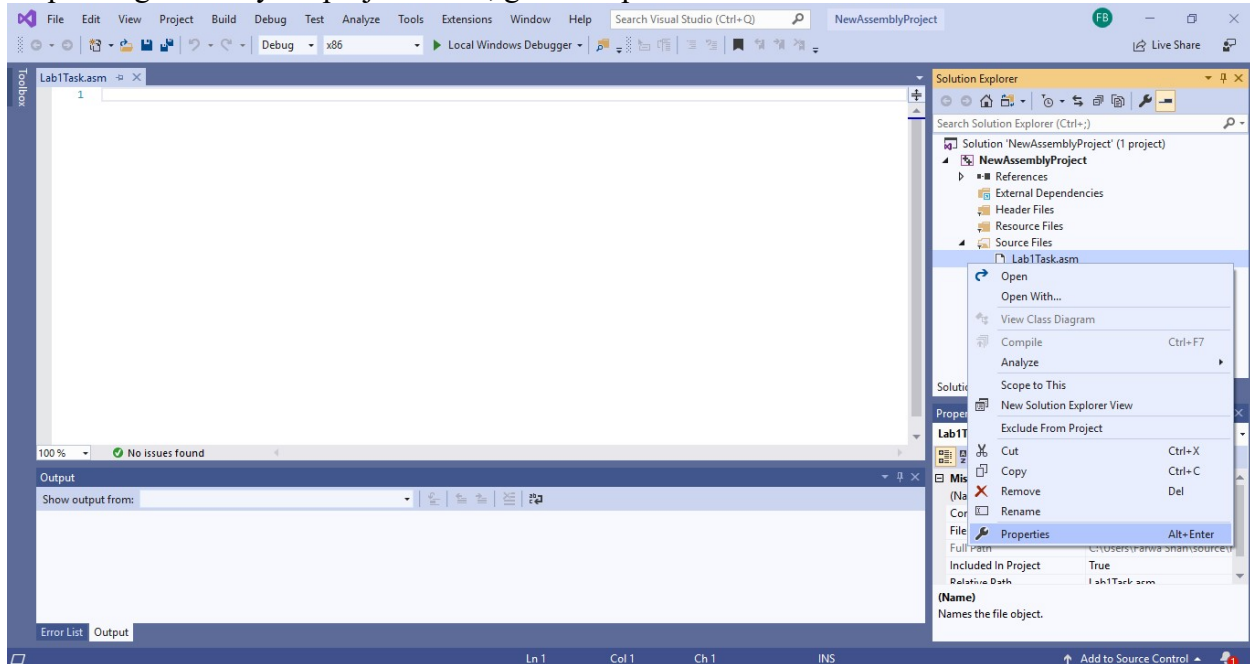
Step 7: Right click on the name of your project-> Build Dependencies-> Build Customizations



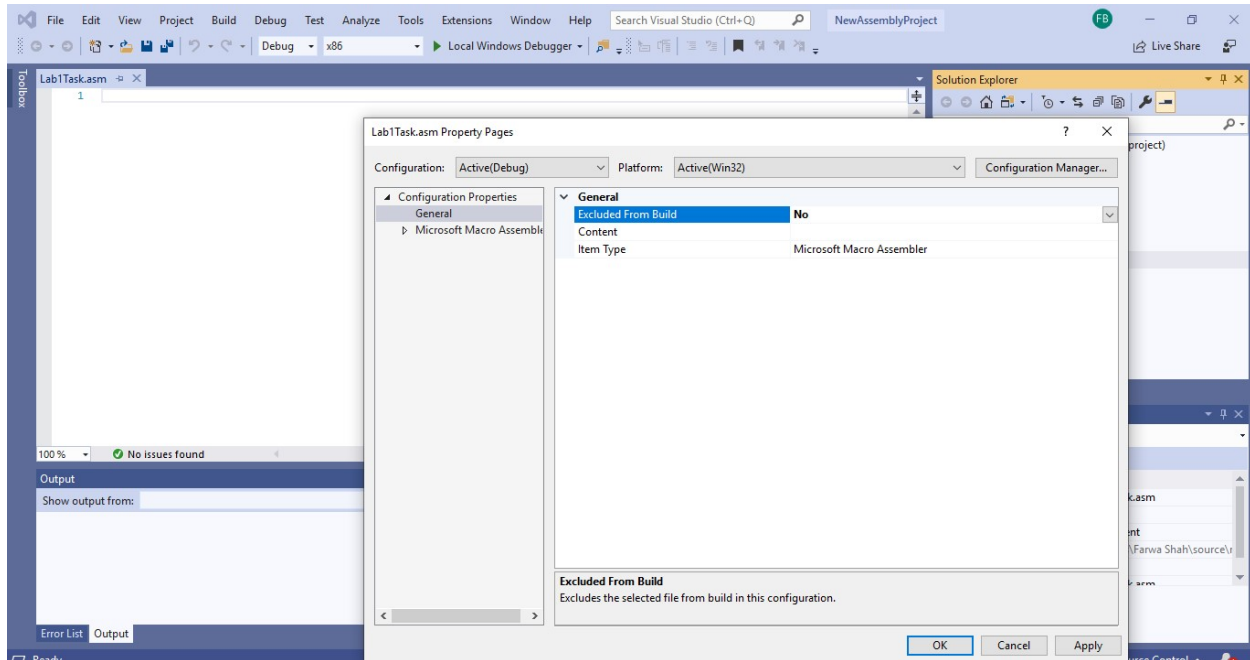
Step 8: Check masm(.targets, .props)



Step 9: Right-click your project name, go to Properties

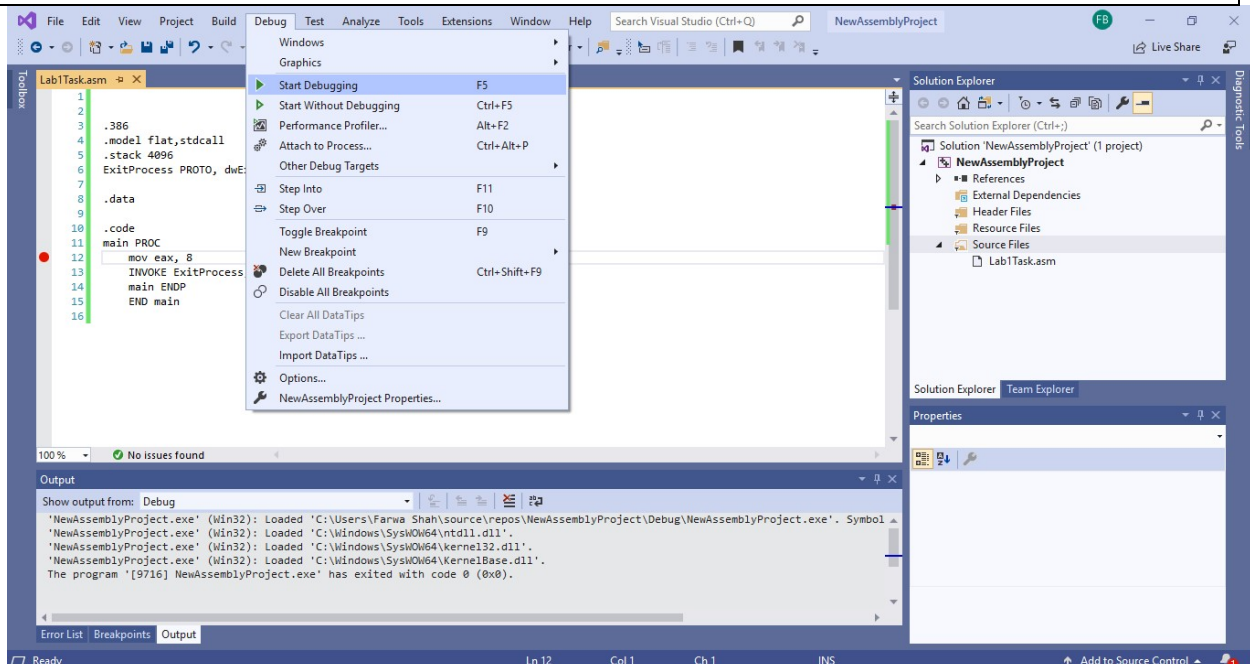


Step 10: from item types select “Microsoft Macro Assemble” and “No” from Exclude From Build



Step 11: Write your first code and click Start Debugging from Debug.

```
.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD
.data
.code
main PROC
    mov eax, 8
    INVOKE ExitProcess, 0
main ENDP
END main
```



Step 12: Click Debug-> Wind-ows-> memory or registers to check authenticity of your code and move forward by clicking the Step-over icon .

